



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

DEPARTAMENTO
DE ELECTRÓNICA
E INFORMÁTICA

Informe Tarea 1: MongoDB

Curso: Bases de Datos Avanzadas INF-325

Profesor: José Luis Martí Lara

Ayudante: Eduardo Rodríguez Prado

Autor: Ignacio Gómez González

Rol: 202273553-8



1. Consultas	3
1.1 Proyección, filtrado y ordenamiento sobre campos simples	3
1.2 Búsqueda dentro de un arreglo simple	4
1.3 Búsqueda sobre arreglos de documentos embebidos	5
1.4 Pipeline de agregación	5
2. Procesamiento de consultas	7
2.1 Ejecución sin índices	7
2.2 Índice sobre Nacionalidad	8
2.3 Índice sobre Edad	9
2.4 Comparación de planes de ejecución	10
2.5 Escalabilidad	11
3. Flexibilidad de esquema, resiliencia y actualización	12
3.1 Flexibilidad de Esquema	12
3.2 Resiliencia de la base de datos	13
3.3 Actualización masiva de documentos	13
4. Cierre	17
5. Fuentes de Datos	17
6. Repositorio GitHub	17
7. Declaración uso de IA	18

1. Consultas

1.1 Proyección, filtrado y ordenamiento sobre campos simples

```
db["Voice-actors"].find(  
  {  
    Edad: {$gt: 50}  
  },  
  {  
    _id: 0,  
    Nombre: 1,  
    Edad: 1,  
    Nacionalidad: 1,  
    Patrimonio: 1  
  }  
)  
.sort({Edad: -1})  
.limit(10)
```

Se busca en la colección “Voice-actors” usando el comando “.find(filtro, proyección)”, donde “filtro” determina qué documentos se seleccionarán y “proyección” determina qué campos de los documentos encontrados se quieren mostrar. En el parámetro del filtro se buscan los actores o actrices de voz mayores de 50 años, luego en el parámetro de proyección se listan 4 campos a mostrar, los cuales son “Nombre”, “Edad”, “Nacionalidad” y “Patrimonio”. Luego se añaden dos funciones para cumplir con las condiciones descritas al inicio del ítem, una es un sort, con el cual se ordenan de manera descendente las edades y el segundo es limitarse a mostrar máximo 10 documentos.

```
< {  
  Nombre: 'Nobuyo Oyama',  
  Nacionalidad: 'Japon',  
  Edad: 72,  
  Patrimonio: 12100000  
}  
{  
  Nombre: 'Unsho Ishizuka',  
  Nacionalidad: 'Japon',  
  Edad: 72,  
  Patrimonio: 6450000  
}  
{  
  Nombre: 'Kujira',  
  Nacionalidad: 'Japon',  
  Edad: 70,  
  Patrimonio: 6050000  
}
```

```
{  
  Nombre: 'TARAKO',  
  Nacionalidad: 'Japon',  
  Edad: 69,  
  Patrimonio: 10600000  
}  
{  
  Nombre: 'Koichi Yamadera',  
  Nacionalidad: 'Japon',  
  Edad: 68,  
  Patrimonio: 13900000  
}  
{  
  Nombre: 'Kotono Mitsuishi',  
  Nacionalidad: 'Japon',  
  Edad: 67,  
  Patrimonio: 12500000  
}
```

```
{  
  Nombre: 'Hikaru Midorikawa',  
  Nacionalidad: 'Japon',  
  Edad: 59,  
  Patrimonio: 13350000  
}  
{  
  Nombre: 'Haruka Terui',  
  Nacionalidad: 'Japon',  
  Edad: 58,  
  Patrimonio: 4300000  
}  
{  
  Nombre: 'Joji Nakata',  
  Nacionalidad: 'Japon',  
  Edad: 58,  
  Patrimonio: 8750000  
}  
{  
  Nombre: 'Kikuko Inoue',  
  Nacionalidad: 'Japon',  
  Edad: 57,  
  Patrimonio: 13650000  
}
```

1.2 Búsqueda dentro de un arreglo simple

Se selecciona de nacionalidad “Japon” y de idioma “Ingles”.

```
db["Voice-actors"].find(
{
  Nacionalidad: "Japon",
  Idiomas: "Ingles"
},
{
  _id: 0,
  Nombre: 1,
  Nacionalidad: 1,
  Idiomas: 1
}
)
.sort({Nombre: 1})
.limit(10)
```

Se busca en la colección “Voice-actors”, usando como filtro la nacionalidad “Japon” e idiomas “Ingles” (MongoDB interpreta que se quiere un documento cuyo array Idiomas contenga "Ingles"), en la proyección se listan 3 campos a mostrar, los cuales son “Nombre”, “Nacionalidad” e “Idiomas”. Luego se añaden dos funciones para cumplir con las condiciones descritas al inicio del ítem, una es un sort, con el cual se ordenan de manera ascendente por nombre (A-Z) y el segundo es limitarse a mostrar máximo 10 documentos.

```
< {
  Nombre: 'Akio Otsuka',
  Nacionalidad: 'Japon',
  Idiomas: [
    'Japones',
    'Ingles'
  ]
}
{
  Nombre: 'Aoi Yuki',
  Nacionalidad: 'Japon',
  Idiomas: [
    'Japones',
    'Ingles'
  ]
}
{
  Nombre: 'Chiwa Saito',
  Nacionalidad: 'Japon',
  Idiomas: [
    'Japones',
    'Ingles'
  ]
}
```

```
{
  Nombre: 'Haruka Terui',
  Nacionalidad: 'Japon',
  Idiomas: [
    'Japones',
    'Ingles'
  ]
}
{
  Nombre: 'Hekiru Shiina',
  Nacionalidad: 'Japon',
  Idiomas: [
    'Japones',
    'Ingles'
  ]
}
{
  Nombre: 'Hiro Shimono',
  Nacionalidad: 'Japon',
  Idiomas: [
    'Japones',
    'Ingles'
  ]
}
```

```
{
  Nombre: 'Ikue Otani',
  Nacionalidad: 'Japon',
  Idiomas: [
    'Japones',
    'Ingles'
  ]
}
{
  Nombre: 'Kikuko Inoue',
  Nacionalidad: 'Japon',
  Idiomas: [
    'Japones',
    'Ingles'
  ]
}
{
  Nombre: 'Masaya Matsukaze',
  Nacionalidad: 'Japon',
  Idiomas: [
    'Japones',
    'Ingles'
  ]
}
{
  Nombre: 'Minami Takayama',
  Nacionalidad: 'Japon',
  Idiomas: [
    'Japones',
    'Ingles'
  ]
}
```

1.3 Búsqueda sobre arreglos de documentos embebidos

```
db["Voice-actors"].find(  
  {  
    Edad: {$lt: 30},  
    Personajes: {  
      $elemMatch: {  
        Rol: "Principal",  
        Generos: "Accion"  
      }  
    }  
  },  
  {  
    _id: 0,  
    Nombre: 1,  
    Edad: 1,  
    Personajes: 1  
  }  
)  
.sort({Edad: 1})  
.limit(10)
```

Se busca en la colección “Voice-actors”, usando como filtro la “Edad” como menor a 30 y luego en “Personajes” buscar al menos un array (\$elemMatch) que cumpla con tener el rol de personaje “Principal” y pertenecer al género de “Accion”. En la proyección se listan 3 campos a mostrar, los cuales son “Nombre”, “Edad” y “Personajes”. Luego se añaden dos funciones para cumplir con las condiciones descritas al inicio del ítem, una es un sort, con el cual se ordenan de manera ascendente por edad y el segundo es limitarse a mostrar máximo 10 documentos.

```
< {  
  Nombre: 'Hana Kim',  
  Edad: 26,  
  Personajes: [  
    {  
      NombrePersonaje: 'Yuna Arai',  
      Produccion: 'Spirit Hunters',  
      TipoProduccion: 'Profesional',  
      Rol: 'Principal',  
      CantidadApariciones: 42,  
      Generos: [  
        'Sobrenatural',  
        'Accion'  
      ]  
    }  
  ]  
}
```

```
{  
  Nombre: 'Sofia Nakamura',  
  Edad: 27,  
  Personajes: [  
    {  
      NombrePersonaje: 'Akari Mizuno',  
      Produccion: 'Eclipse Academy',  
      TipoProduccion: 'Profesional',  
      Rol: 'Principal',  
      CantidadApariciones: 24,  
      Generos: [  
        'Accion',  
        'Fantasia'  
      ]  
    }  
  ]  
}
```

1.4 Pipeline de agregación

```
db["Voice-actors"].aggregate([
  {
    $sort: {Patrimonio: -1}
  },
  {
    $skip: 4
  },
  {
    $limit: 10
  },
  {
    $project: {
      _id: 0,
      Nombre: 1,
      Nacionalidad: 1,
      Edad: 1,
      Patrimonio: 1
    }
  }
])
```

En este caso se usa aggregate que funciona como una ejecución de una secuencia de etapas, entonces primero como dice el enunciado de la tarea se ordena de acuerdo al patrimonio, en este caso se ordenan de forma descendente y luego se saltan los primeros 4 (del 1 al 4), seguido de obtener los siguientes 10 registros desde la posición 5. Finalmente con project se hace la proyección de los cuatro campos “Nombre”, “Nacionalidad”, “Edad” y “Patrimonio”.

```
< {
  Nombre: 'Maaya Sakamoto',
  Nacionalidad: 'Japon',
  Edad: 35,
  Patrimonio: 17350000
}
{
  Nombre: 'Minorí Suzuki',
  Nacionalidad: 'Japon',
  Edad: 27,
  Patrimonio: 17300000
}
{
  Nombre: 'Rina Sato',
  Nacionalidad: 'Japon',
  Edad: 39,
  Patrimonio: 17200000
}
```

```
{
  Nombre: 'Akio Otsuka',
  Nacionalidad: 'Japon',
  Edad: 56,
  Patrimonio: 16450000
}
{
  Nombre: 'Megumi Ogata',
  Nacionalidad: 'Japon',
  Edad: 42,
  Patrimonio: 16150000
}
{
  Nombre: 'Chiwa Saito',
  Nacionalidad: 'Japon',
  Edad: 52,
  Patrimonio: 16050000
}
```

```
{
  Nombre: 'Aimi Tanaka',
  Nacionalidad: 'Japon',
  Edad: 26,
  Patrimonio: 14900000
}
{
  Nombre: 'Kazuya Nakai',
  Nacionalidad: 'Japon',
  Edad: 33,
  Patrimonio: 14350000
}
{
  Nombre: 'Nana Mizuki',
  Nacionalidad: 'Japon',
  Edad: 33,
  Patrimonio: 14300000
}
{
  Nombre: 'Koichi Yamadera',
  Nacionalidad: 'Japon',
  Edad: 68,
  Patrimonio: 13900000
}
```

2. Procesamiento de consultas

2.1 Ejecución sin índices

Primera consulta:

```
db["Voice-actors"].find(
  {
    Edad: {$gt: 50}
  },
  {
    _id: 0,
    Nombre: 1,
    Edad: 1,
    Nacionalidad: 1,
    Patrimonio: 1
  }
)
.sort({Edad: -1})
.limit(10)
.explain("executionStats")
```

Segunda consulta:

```
db["Voice-actors"].find(
  {
    Nacionalidad: "Japon",
    Idiomas: "Ingles"
  },
  {
    _id: 0,
    Nombre: 1,
    Nacionalidad: 1,
    Idiomas: 1
  }
)
.sort({Nombre: 1})
.limit(10)
.explain("executionStats")
```

Tercera consulta:

```
db["Voice-actors"].find(
  {
    Edad: {$lt: 30},
    Personajes: {
      $elemMatch: {
        Rol: "Principal",
        Generos: "Accion"
      }
    }
  },
  {
    _id: 0,
    Nombre: 1,
    Edad: 1,
    Personajes: 1
  }
)
.sort({Edad: 1})
```

```
.limit(10)
.explain("executionStats")
```

Cuarta consulta:

```
db["Voice-actors"].explain("executionStats").aggregate([
  {
    $sort: {Patrimonio: -1}
  },
  {
    $skip: 4
  },
  {
    $limit: 10
  },
  {
    $project: {
      _id: 0,
      Nombre: 1,
      Nacionalidad: 1,
      Edad: 1,
      Patrimonio: 1
    }
  }
])
```

Estudiando la salida con ChatGPT 5.6 sol, pude ver que en común las cuatro columnas aparecen con "COLLSCAN", lo que quiere decir que se recorre la colección completa.

#Consulta	Plan	Docs Examinados	Keys Examinadas	Retornados	Tiempo
1	COLLSCAN	84	0	10	2 ms
2	COLLSCAN	84	0	10	1 ms
3	COLLSCAN	84	0	2	1 ms
4	COLLSCAN	84	0	10	1 ms

2.2 Índice sobre Nacionalidad

Primero se crea el índice con

```
db["Voice-actors"].createIndex({Nacionalidad: 1})
```

Luego se ejecuta nuevamente la consulta

```
db["Voice-actors"].find(
  {
    Nacionalidad: "Japon",
    Idiomas: "Ingles"
  },
  {
    _id: 0,
    Nombre: 1,
    Nacionalidad: 1,
    Idiomas: 1
  }
)
```

```
}  
)  
.sort({Nombre: 1})  
.limit(10)  
.explain("executionStats")
```

Ahora se tiene "IXSCAN" para recorrer en base al índice en lugar de recorrer toda la colección a diferencia del anterior resultado.

Métrica	Sin Índice	Con Índice
Plan	COLLSCAN	IXSCAN
Docs examinados	84	73
Keys examinadas	0	73
Retornados	10	10
Tiempo	1 ms	1 ms

No hay una mejora temporal visible porque la colección es muy pequeña y, además, Nacionalidad = "Japon" coincide en casi todos los documentos.

2.3 Índice sobre Edad

Primero se crea el índice con

```
db["Voice-actors"].createIndex({Edad: 1})
```

Luego se ejecuta nuevamente la consulta

```
db["Voice-actors"].find(  
  {  
    Edad: {$gt: 50}  
  },  
  {  
    _id: 0,  
    Nombre: 1,  
    Edad: 1,  
    Nacionalidad: 1,  
    Patrimonio: 1  
  }  
)  
.sort({Edad: -1})  
.limit(10)  
.explain("executionStats")
```

Y la consulta

```
db["Voice-actors"].find(  
  {  
    Edad: {$lt: 30},  
    Personajes: {  
      $elemMatch: {  
        Rol: "Principal",  
        Generos: "Accion"  
      }  
    }  
  })
```

```
    }  
  }  
},  
{  
  _id: 0,  
  Nombre: 1,  
  Edad: 1,  
  Personajes: 1  
}  
)  
.sort({Edad: 1})  
.limit(10)  
.explain("executionStats")
```

Luego comparativamente tenemos para la primera consulta

Métrica	Sin Índice	Con Índice
Plan	COLLSCAN	IXSCAN
Docs examinados	84	10
Keys examinadas	0	10
Retornados	10	10
Tiempo	1 ms	0 ms

Y para la otra consulta

Métrica	Sin Índice	Con Índice
Plan	COLLSCAN	IXSCAN
Docs examinados	84	8
Keys examinadas	0	8
Retornados	2	2
Tiempo	1 ms	0 ms

El índice sobre Edad presentó el mayor beneficio en comparación al índice de “Nacionalidad”, reduciendo considerablemente la cantidad de documentos examinados, esto es principalmente debido a que la mayoría de los registros poseen nacionalidad japonesa. En general, la efectividad de un índice depende de qué tan selectivo sea el atributo consultado.

2.4 Comparación de planes de ejecución

Al comparar los resultados obtenidos mediante “explain(“executionStats”)” antes y después de crear los índices, se observa un cambio en la forma en que MongoDB procesa las consultas. Inicialmente, las consultas utilizaron “COLLSCAN”, lo que implica recorrer los documentos de la colección para determinar cuáles cumplen con las condiciones. En los

cuatro casos se examinaron los 84 documentos existentes y no se examinó ninguna clave de índice.

Luego de crear el índice sobre "Nacionalidad", la consulta que utiliza este campo cambió de "COLLSCAN" a "IXSCAN". La cantidad de documentos examinados disminuyó de 84 a 73, mientras que se examinaron 73 claves del índice y se retornaron los mismos 10 documentos. El tiempo reportado se mantuvo en 1 ms. La mejora fue limitada debido a que Nacionalidad = "Japon" coincide con una gran proporción de los documentos, por lo que el índice posee una baja selectividad para esta consulta.

El índice sobre "Edad" produjo un cambio más significativo. Para la consulta Edad > 50, el plan cambió de "COLLSCAN" a "IXSCAN", reduciendo los documentos examinados de 84 a 10, examinando 10 claves y retornando los mismos 10 documentos. El tiempo de ejecución reportado disminuyó de 1 ms a 0 ms. Para la consulta Edad < 30, también se utilizó "IXSCAN", reduciendo los documentos examinados de 84 a 8, con 8 claves examinadas y los mismos 2 documentos retornados, mientras que el tiempo pasó de 1 ms a 0 ms.

Por lo tanto, los índices permitieron a MongoDB seleccionar planes de ejecución que utilizan "IXSCAN" en lugar de "COLLSCAN", reduciendo la cantidad de documentos que deben ser examinados. Los documentos retornados no cambiaron, ya que los índices no modifican el resultado de las consultas, sino la forma en que MongoDB accede a los datos. Aunque las diferencias de tiempo son pequeñas debido al reducido tamaño de la colección, la disminución en documentos examinados permite observar claramente el beneficio de los índices. Además, los resultados muestran que la utilidad de un índice depende de la selectividad del campo, siendo "Edad" más efectivo que "Nacionalidad" para los datos utilizados. Por lo tanto, MongoDB seleccionó como plan ganador aquellos que utilizan "IXSCAN" cuando los índices creados podían ser aprovechados, en lugar de los planes basados en "COLLSCAN" utilizados inicialmente.

2.5 Escalabilidad

Si la colección aumentara desde 84 documentos hasta 10 mil millones, las diferencias observadas entre consultas con y sin índices serían mucho más importantes. Una consulta ejecutada mediante "COLLSCAN" podría llegar a examinar todos los documentos de la colección. En las pruebas realizadas, las consultas sin índices examinaron los 84 documentos disponibles, por lo que al aumentar el volumen de datos este comportamiento dejaría de ser prácticamente despreciable y podría requerir una gran cantidad de tiempo, CPU, memoria y operaciones de entrada/salida.

En cambio, las consultas que utilizan índices pueden reducir considerablemente la cantidad de documentos examinados. En la consulta sobre "Edad > 50", el uso del índice redujo los documentos examinados de 84 a 10. Además, como el índice permite obtener los registros ordenados por edad y la consulta posee un límite de 10 resultados, MongoDB puede detener la búsqueda una vez encontrados esos 10 documentos. Por lo tanto, incluso con miles de millones de registros, si existen suficientes coincidencias, la cantidad examinada podría mantenerse muy reducida en comparación con un recorrido completo.

En la consulta sobre “Edad < 30”, el índice redujo los documentos examinados de 84 a 8 antes de evaluar la condición asociada a “Personajes”. Estos 8 documentos representan aproximadamente un 9,5 % de la colección actual. Debido al límite de 10 resultados, MongoDB podría finalizar antes si encuentra suficientes documentos que también satisfagan la condición sobre “Personajes”.

El índice sobre “Nacionalidad” produjo una reducción menor: se pasó de 84 a 73 documentos examinados, debido a que la mayoría de los registros poseen nacionalidad “Japon”. Esto corresponde aproximadamente al 86,9 % de la colección. Si esa proporción se mantuviera en 10 mil millones de documentos, una consulta por nacionalidad japonesa podría involucrar alrededor de 8.690 millones de entradas del índice y documentos candidatos, por lo que el beneficio respecto de un “COLLSCAN” sería bastante menor. Esto demuestra que no basta con crear un índice, también es importante que el atributo tenga una buena capacidad para discriminar entre los registros.

Los índices también requieren almacenamiento adicional. A diferencia de una consulta sin índices, MongoDB debe mantener estructuras auxiliares que contienen los valores indexados y referencias a los documentos correspondientes. Con 10 mil millones de documentos, los índices sobre “Edad” y “Nacionalidad” podrían ocupar una cantidad considerable de almacenamiento, además de aumentar el costo de las operaciones de inserción, eliminación y actualización, ya que dichas estructuras deben mantenerse actualizadas.

En conclusión, los resultados obtenidos con “explain()” muestran que el diseño de índices se vuelve fundamental cuando aumenta el volumen de datos. Un “COLLSCAN” que actualmente examina solo 84 documentos podría llegar a examinar hasta 10 mil millones, mientras que un índice adecuado puede restringir considerablemente el conjunto de documentos candidatos e incluso permitir detener la búsqueda tempranamente gracias a operaciones como “limit()”. Sin embargo, el beneficio depende directamente de la selectividad del campo indexado, como se observó al comparar los índices de “Edad” y “Nacionalidad”.

3. Flexibilidad de esquema, resiliencia y actualización

3.1 Flexibilidad de Esquema

- **¿MongoDB permitiría insertar el documento?** Sí, MongoDB permitiría insertar el documento aunque no contenga el atributo Fecha de nacimiento, ya que por defecto utiliza un esquema flexible y los documentos pertenecientes a una misma colección no están obligados a poseer exactamente los mismos atributos.
- **¿Qué consecuencias podría tener posteriormente?** La ausencia del atributo podría provocar inconsistencias en los datos y dificultades en consultas u operaciones que dependan de la fecha de nacimiento. Por ejemplo, una consulta

que filtre utilizando este campo no podría evaluar de la misma manera al documento que no lo posee. También podría dificultar procesos posteriores que necesiten utilizar la fecha de nacimiento para calcular o verificar otros datos, como la edad.

- **¿Considera conveniente permitir documentos con atributos faltantes en este caso?** No sería conveniente para atributos considerados fundamentales, como Fecha de nacimiento. Aunque la flexibilidad de esquema permite almacenar documentos diferentes, mantener una estructura consistente para los datos principales facilita las consultas, actualizaciones y procesamiento posterior. Los atributos opcionales sí podrían aprovechar esta flexibilidad.
- **¿Existe alguna forma de controlar o validar la estructura de los documentos en MongoDB?** Sí, MongoDB permite establecer reglas de validación para una colección mediante Schema Validation. Por ejemplo, utilizando \$jsonSchema se pueden definir campos obligatorios mediante required y establecer los tipos de datos que debe poseer cada atributo. De esta forma, se podría exigir la presencia de Fecha de nacimiento y rechazar documentos que no cumplan con la estructura definida.

3.2 Resiliencia de la base de datos

- **¿Funcionó la consulta?** Sí, la consulta pudo ejecutarse luego de detener el nodo 3 que originalmente cumplía el rol de primario.
- **¿Por qué funcionó la consulta?** La consulta funcionó debido a que los nodos de MongoDB se encuentran configurados como un Replica Set. Al dejar de estar disponible el nodo primario, los nodos restantes pueden realizar una elección automática y uno de los secundarios pasa a asumir el rol de nuevo primario (en este caso el nodo 1 pasó a ser el primario). Como los nodos secundarios mantienen copias replicadas de los datos, la caída de un nodo no implica necesariamente que la base de datos deje de estar disponible. Una vez elegido el nuevo primario, el sistema puede continuar atendiendo las operaciones, proporcionando tolerancia a fallos y alta disponibilidad.

3.3 Actualización masiva de documentos

Se ejecuta antes de la actualización

```
db["Voice-actors"].find(
{
  $or: [
    { Edad: { $lt: 40 } },
    { Edad: { $gte: 40 } }
  ]
},
{
  _id: 0,
  Nombre: 1,
  Edad: 1,
  Patrimonio: 1,
  Personajes: 1
}
```

).limit(5)

```
<{
  Nombre: 'Haruka Ishida',
  Edad: 24,
  Patrimonio: 17600000,
  Personajes: [
    {
      NombrePersonaje: 'Miu Iruma',
      Produccion: 'Danganronpa V3: Killing Harmony',
      TipoProduccion: 'Profesional',
      Rol: 'Secundario',
      CantidadApariciones: 38,
      Generos: [
        'Misterio',
        'Aventura',
        'Novela visual'
      ]
    }
  ]
}
```

```
{
  Nombre: 'Shunsuke Takeuchi',
  Edad: 25,
  Patrimonio: 9900000,
  Personajes: [
    {
      NombrePersonaje: 'Gonta Gokuhara',
      Produccion: 'Danganronpa V3: Killing Harmony',
      TipoProduccion: 'Profesional',
      Rol: 'Secundario',
      CantidadApariciones: 24,
      Generos: [
        'Misterio',
        'Aventura',
        'Novela visual'
      ]
    },
    {
      NombrePersonaje: 'Einar',
      Produccion: 'Vinland Saga',
      TipoProduccion: 'Profesional',
      Rol: 'Principal',
      CantidadApariciones: 20,
      Generos: [
        'Drama',
        'Historico'
      ]
    }
  ]
}
```

```
{
  Nombre: 'Hana Kim',
  Edad: 26,
  Patrimonio: 850000,
  Personajes: [
    {
      NombrePersonaje: 'Yuna Arai',
      Produccion: 'Spirit Hunters',
      TipoProduccion: 'Profesional',
      Rol: 'Principal',
      CantidadApariciones: 42,
      Generos: [
        'Sobrenatural',
        'Accion'
      ]
    }
  ]
}
```

```
{
  Nombre: 'Sora Tokui',
  Edad: 26,
  Patrimonio: 8850000,
  Personajes: [
    {
      NombrePersonaje: 'Tenko Chabashira',
      Produccion: 'Danganronpa V3: Killing Harmony',
      TipoProduccion: 'Profesional',
      Rol: 'Secundario',
      CantidadApariciones: 33,
      Generos: [
        'Misterio',
        'Aventura',
        'Novela visual'
      ]
    }
  ]
}
```

```
{
  Nombre: 'Aimi Tanaka',
  Edad: 26,
  Patrimonio: 14900000,
  Personajes: [
    {
      NombrePersonaje: 'Himiko Yumeno',
      Produccion: 'Danganronpa V3: Killing Harmony',
      TipoProduccion: 'Profesional',
      Rol: 'Secundario',
      CantidadApariciones: 32,
      Generos: [
        'Misterio',
        'Aventura',
        'Novela visual'
      ]
    },
    {
      NombrePersonaje: 'Umaru Doma',
      Produccion: 'Himouto! Umaru-chan',
      TipoProduccion: 'Profesional',
      Rol: 'Principal',
      CantidadApariciones: 17,
      Generos: [
        'Comedia',
        'Slice of Life'
      ]
    }
  ]
}
```

Ahora se procede a actualizar masivamente con

```
db["Voice-actors"].bulkWrite([
  {
    updateMany: {
      filter: { Edad: { $lt: 40 } },
      update: {
        $push: {
          Personajes: {
            Nombre: "Sansa",
            Produccion: "Sansa Ball Race",
            TipoProduccion: "Profesional",
            Rol: "Principal",
            Apariciones: 3,
```

```
        Generos: ["Animacion"]
      }
    },
    $mul: {
      Patrimonio: 1.32
    }
  }
},
{
  updateMany: {
    filter: { Edad: { $gte: 40 } },
    update: {
      $push: {
        Personajes: {
          Nombre: "Sansa",
          Produccion: "Sansa Ball Race",
          TipoProduccion: "Profesional",
          Rol: "Secundario",
          Apariciones: 1,
          Generos: ["Animacion"]
        }
      }
    },
    $mul: {
      Patrimonio: 1.32
    }
  }
}
])
```

Para realizar la actualización se utilizó “bulkWrite()” junto con dos operaciones “updateMany()”. La primera operación selecciona a todos los actores menores de 40 años y la segunda a aquellos de 40 años o más. De esta manera, es posible aplicar las características correspondientes al nuevo personaje según la edad sin modificar manualmente cada documento.

En ambas operaciones se utiliza “\$push” para agregar el nuevo personaje al arreglo “Personajes” y “\$mul” con un factor de “1.32” para incrementar el patrimonio en un 32 %. Esta solución permite realizar la modificación de forma masiva sobre toda la colección mediante únicamente dos operaciones de actualización.

```
< {
  acknowledged: true,
  insertedCount: 0,
  insertedIds: {},
  matchedCount: 84,
  modifiedCount: 84,
  deletedCount: 0,
  upsertedCount: 0,
  upsertedIds: {}
}
```

Luego después de la actualización al realizar la consulta anterior se obtiene

<pre>{ Nombre: 'Haruka Ishida', Edad: 24, Patrimonio: 23232000, Personajes: [{ NombrePersonaje: 'Miu Iruma', Producción: 'Danganronpa V3: Killing Harmony', TipoProducción: 'Profesional', Rol: 'Secundario', CantidadApariciones: 38, Generos: ['Misterio', 'Aventura', 'Novela visual'] }, { Nombre: 'Sansa', Producción: 'Sansa Ball Race', TipoProducción: 'Profesional', Rol: 'Principal', Apariciones: 3, Generos: ['Animación'] }] }</pre>	<pre>{ Nombre: 'Shunsuke Takeuchi', Edad: 35, Patrimonio: 19668000, Personajes: [{ NombrePersonaje: 'Gonta Gokuhara', Producción: 'Danganronpa V3: Killing Harmony', TipoProducción: 'Profesional', Rol: 'Secundario', CantidadApariciones: 24, Generos: ['Misterio', 'Aventura', 'Novela visual'] }, { NombrePersonaje: 'Einar', Producción: 'Vinland Saga', TipoProducción: 'Profesional', Rol: 'Principal', CantidadApariciones: 20, Generos: ['Drama', 'Historico'] }, { Nombre: 'Sansa', Producción: 'Sansa Ball Race', TipoProducción: 'Profesional', Rol: 'Principal', Apariciones: 3, Generos: ['Animación'] }] }</pre>	<pre>{ Nombre: 'Hana Kim', Edad: 26, Patrimonio: 1122000, Personajes: [{ NombrePersonaje: 'Yuna Arai', Producción: 'Spirit Hunters', TipoProducción: 'Profesional', Rol: 'Principal', CantidadApariciones: 42, Generos: ['Sobrenatural', 'Acción'] }, { Nombre: 'Sansa', Producción: 'Sansa Ball Race', TipoProducción: 'Profesional', Rol: 'Principal', Apariciones: 3, Generos: ['Animación'] }] }</pre>
<pre>{ Nombre: 'Sora Tokui', Edad: 26, Patrimonio: 11682000, Personajes: [{ NombrePersonaje: 'Tenko Chabashira', Producción: 'Danganronpa V3: Killing Harmony', TipoProducción: 'Profesional', Rol: 'Secundario', CantidadApariciones: 33, Generos: ['Misterio', 'Aventura', 'Novela visual'] }, { Nombre: 'Sansa', Producción: 'Sansa Ball Race', TipoProducción: 'Profesional', Rol: 'Principal', Apariciones: 3, Generos: ['Animación'] }] }</pre>	<pre>{ Nombre: 'Aiai Tanaka', Edad: 26, Patrimonio: 19668000, Personajes: [{ NombrePersonaje: 'Hinako Yumeno', Producción: 'Danganronpa V3: Killing Harmony', TipoProducción: 'Profesional', Rol: 'Secundario', CantidadApariciones: 32, Generos: ['Misterio', 'Aventura', 'Novela visual'] }, { NombrePersonaje: 'Umaru Doma', Producción: 'Himouto! Umaru-chan', TipoProducción: 'Profesional', Rol: 'Principal', CantidadApariciones: 17, Generos: ['Comedia', 'Slice of Life'] }, { Nombre: 'Sansa', Producción: 'Sansa Ball Race', TipoProducción: 'Profesional', Rol: 'Principal', Apariciones: 3, Generos: ['Animación'] }] }</pre>	

Como se observa en los resultados posteriores a la actualización, el personaje Sansa, perteneciente a la producción Sansa Ball Race, fue agregado al arreglo Personajes de los documentos. Para los actores menores de 40 años se asignó el rol Principal con 3 apariciones, mientras que para aquellos de 40 años o más se asignó el rol Secundario con 1 aparición. Además, el valor de Patrimonio fue incrementado en un 32 %.

La actualización se realizó mediante bulkWrite() utilizando dos operaciones updateMany(), permitiendo modificar masivamente los documentos de la colección sin realizar cambios individuales.

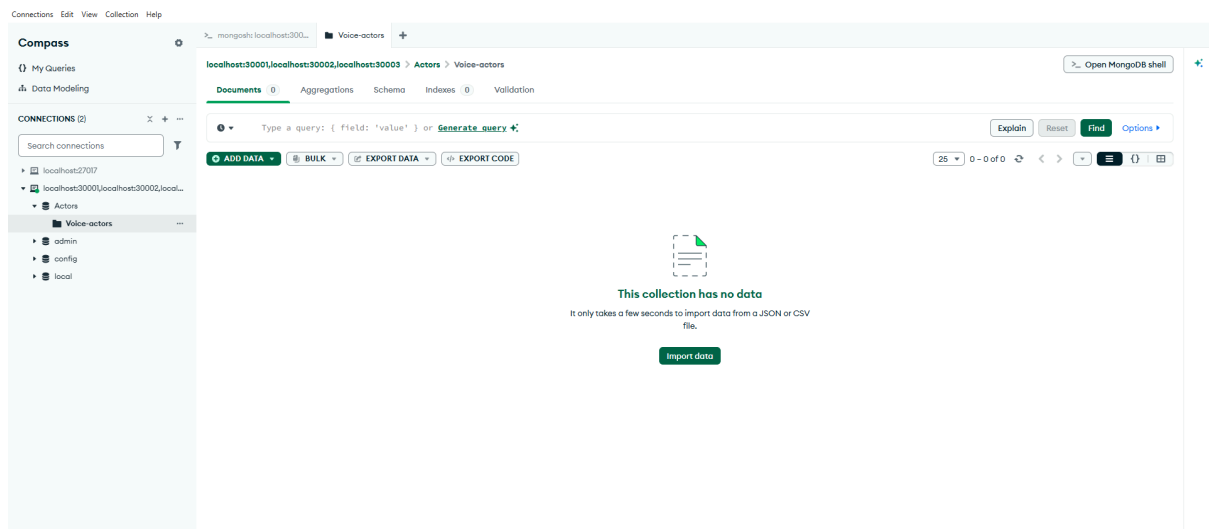
4. Cierre

```
> _MONGOSH

> use Actors
< already on db Actors

> db.dropDatabase()
< { ok: 1, dropped: 'Actors' }

my-replica-set [primary] Actors>|
```



5. Fuentes de Datos

Los datos fueron generados con ChatGPT 5.6 Sol.

6. Repositorio GitHub

<https://github.com/Neichoo/Tareas-BaseDeDatosAvanzadas>

Dentro de la carpeta “Tarea 1 MongoDB” se encuentra todo lo desarrollado para esta tarea, al entrar, se podrá ver:

- “README.md” con todas las consideraciones tomadas a la hora de realizar la tarea y también lo necesario para la ejecución de los códigos.
- La carpeta “Imágenes” con toda la evidencia de las ejecuciones de las consultas.



7. Declaración uso de IA

Se hace uso de ChatGPT 5.6 sol para generar los datos y también para corregir y apoyar en la redacción del documento.
